Docker Commands in Detail

A section-by-section reference to the most common Docker commands: what each one does, useful flags, and the gotchas worth knowing.

Images

`docker pull nginx:latest` Downloads an image's layers from a registry. The full form is `registry/namespace/repo:tag`; Docker Hub is assumed if you omit the registry. Omitting the tag means `latest`, which is just a default tag name and not necessarily the newest version. For exact reproducibility, pull by digest: `docker pull nginx@sha256:...`.

`docker images` (same as `docker image ls`) Shows REPOSITORY, TAG, IMAGE ID, CREATED, and SIZE. Useful variants:

- `docker images -a` includes intermediate layers.
- `docker images -f dangling=true` shows untagged leftovers from rebuilds.

`docker build -t myapp:1.0 .` Useful extra flags:

- `-f path/Dockerfile` uses a Dockerfile with a different name or location.
- `--no-cache` ignores cached layers and rebuilds everything.
- `--build-arg VERSION=2` passes a value to an `ARG` in the Dockerfile.
- `--target build` stops at a named stage in a multi-stage build.

`docker tag myapp:1.0 user/myapp:1.0` Adds another name to the same image. Nothing is copied, and both names share one image ID. You need the `user/` or registry prefix before pushing.

`docker push user/myapp:1.0` Uploads the image to a registry. You must be logged in first, and only layers the registry doesn't already have are uploaded.

`docker rmi myapp:1.0` Removes a tag. If the image has other tags, only that tag goes away. If it's the last tag, the layers are deleted. It fails if any container (even a stopped one) uses the image, unless you add `-f`.

`docker history myapp` Lists each layer with its size and the instruction that created it. This is the quickest way to find which `RUN` line bloated your image. Add `--no-trunc` to see full commands.

`docker save -o app.tar myapp` / **`docker load -i app.tar`** Exports and imports an image as a tarball, including all layers and tags. This is for moving images to machines with no registry access (air-gapped servers, for example). Don't confuse it with `docker export` / `import`, which work on a container's flattened filesystem and lose image metadata like `CMD` and `ENV`.

Running containers

- **`docker run nginx`** runs in the foreground and attaches your terminal. Ctrl+C stops it.
- **`docker run --rm alpine echo hi`** replaces the image's default command (`CMD`) with `echo hi`. Anything after the image name is treated as the command to run.
- **`--memory=512m --cpus=1`**: the memory cap is hard (the process is OOM-killed if it exceeds it), while the CPU cap just throttles.
- **`--restart`** options are `no` (default), `on-failure[:max]`, `always`, and `unless-stopped`.

Managing containers

`docker start web` restarts a stopped container with its original configuration (ports, volumes, env). It does not create a new one. Add `-a` to attach to its output.

`docker restart web` is stop followed by start, with the same 10-second grace period.

`docker pause web` / **`docker unpause web`** freezes all processes in the container using the cgroup freezer. Unlike `stop`, no signal is sent, so the app isn't aware it was paused. It's useful for taking a consistent snapshot or temporarily freeing CPU.

Inspecting and debugging

`docker stats` is a live-updating table of CPU %, memory usage/limit, network I/O, and block I/O for running containers. Use `--no-stream` for a single snapshot.

`docker top web` lists the processes running inside the container, as seen from the host. It takes `ps` options, such as `docker top web aux`.

`docker cp web:/app/file.txt .` copies files between a container and your host, in either direction. It works on stopped containers too, which helps when retrieving logs from a crashed one.

`docker diff web` shows filesystem changes since the container started:

- `A` means added
- `C` means changed
- `D` means deleted

It's a good way to spot where an app is writing data that should be on a volume.

Volumes

- **`docker volume create mydata`** creates a named volume. Docker also creates one automatically if you mount a name that doesn't exist.
- **`docker volume ls`** lists all volumes.
- **`docker volume inspect mydata`** shows the driver and the `Mountpoint`, typically `/var/lib/docker/volumes/mydata/_data` on Linux. On Docker Desktop (Mac/Windows) that path is inside a hidden VM.
- **`docker volume rm mydata`** deletes the volume and its data permanently. It fails if a container is still using it.

Networks

- **`docker network ls`** shows the three defaults (`bridge`, `host`, `none`) plus any you've created.
- **`docker network inspect mynet`** shows the subnet, gateway, and which containers are attached with their IPs.
- **`docker network connect mynet web`** attaches a running container to an additional network without restarting it. A container can belong to several networks, which is useful for isolating a database from the public-facing network.
- **`docker network disconnect mynet web`** removes that attachment.

Docker Compose

The modern command is `docker compose` (a plugin). The older standalone `docker-compose` is deprecated.

- **`up -d`** creates and starts everything. Add `--build` to rebuild images first.

- **`down`** removes containers and networks but keeps volumes. **`down -v`** also deletes volumes and their data.
- **`ps`** shows the status of the project's services.
- **`logs -f`** follows combined logs. Add a service name (`logs -f web`) to filter.
- **`build`** rebuilds service images without starting anything.
- **`exec web sh`** opens a shell in a running service (the Compose equivalent of `docker exec`, using the service name rather than the container name).
- **`restart`** restarts services without recreating them. If you changed the Compose file, use `up -d` instead so changes are applied.

Cleanup

- **`docker system df`** shows disk used by images, containers, volumes, and build cache, and how much is reclaimable. Run it first, and add `-v` for a per-item breakdown.
- **`container prune`** deletes all stopped containers.
- **`image prune -a`** deletes every image not used by an existing container.
- **`volume prune`** deletes volumes not attached to any container. This is the riskiest one, since it can wipe database data from a stack you merely stopped.
- **`system prune -a`** combines the above for containers, networks, and images. Volumes are only included if you add `--volumes`.

All of these ask for confirmation unless you pass `-f`.

System and registry

- **`docker version`** prints Client and Server (daemon) versions. If the Server section is missing or errors, the daemon isn't running or you lack permission.
- **`docker info`** gives system-wide details: storage driver, cgroup version, container and image counts, root directory, and registry settings. It's the first thing to check when troubleshooting a setup.
- **`docker login`** authenticates to a registry (Docker Hub by default; add a hostname for others). Credentials are stored in `~/.docker/config.json`, ideally via a credential helper rather than as plain base64. For CI, use an access token instead of your password.
- **`docker logout`** removes the stored credentials.

- **`docker search nginx`** queries Docker Hub from the terminal. Results are limited, so filter with `--filter is-official=true` or just use the Hub website.